

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.1: The Agent Opportunity — From Chatbox to Agent Swarms

● Zhigang Feng

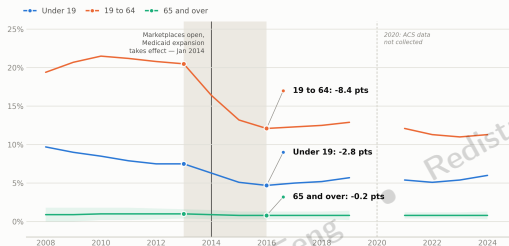
2026-09-13 15:51:23

From an Empty Folder to a Figure

Who gained health coverage after 2013? An empty folder, four prompts, and 22 minutes of agent time — including the one decision the agent stopped to ask for.

Working-age adults gained the most coverage after 2014

Uninsured rate by age group, United States, 2008–2024. Labels show the percentage-point change from 2013 to 2016, when ACA Marketplace and Medicaid expansion coverage took effect. Shaded band = 90% confidence margin of error.



Source: U.S. Census Bureau, American Community Survey 1-year estimates, 2008–2024 (HC-4/5/6, ACS historical tables). "19 to 64" and "65 and over" are derived from the published age groups; see SOURCE5.md for method. Retrieved 2026-09-13.

The agent's figure (ACS, 2008–2024). Lec10 T3 takes the question further.

What the agent did

- Found the right Census table, then hit a wall: the data API now requires a key that is emailed to a person
- Tried the no-key tables, patched a spreadsheet-parsing bug — and stopped to ask: wait for a key, or accept gaps? (11 minutes, 59 tool calls)
- Given a decision, built three consistent age groups with margins of error in under 4 minutes; then the figure and the labels

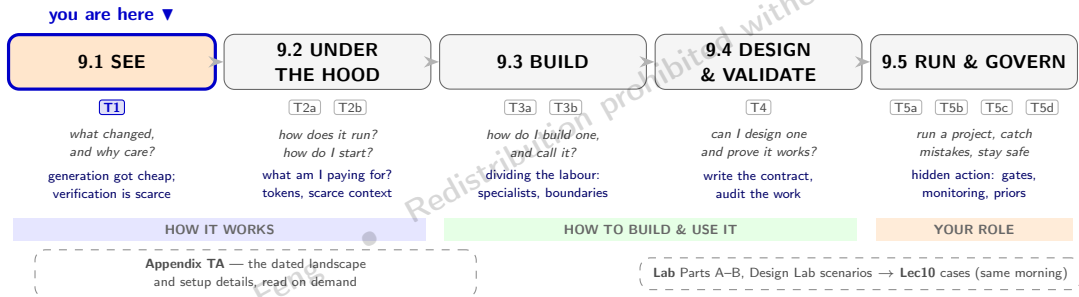
What stayed with the economist

- Choosing the data: it assumed one year's age groups hold in every year (they were redrawn in 2017), and its fallback crossed a survey redesign
- The question, what to mark — and one catch in the final figure: 2020 ACS data were collected; only the standard estimates were not released

Recorded 13 Sep 2026 (Claude Code 2.1.270); transcript in the lab's transcripts/; numbers re-checked against Census tables. [solid]

Lecture 9 in One Map

One lecture, five parts, eleven decks, three questions: how agentic AI works, how to use it well, and what your role is.



Two research threads run through all five parts: **HA-Ramsey** (a published research program, full case in Lec10 T8) and the **ES Fellows dataset** (a project you could build this week, full case in Lec10 T4). Here we study the workflows; the cases themselves come later this morning. Under each part, in blue, is the economist's version of its question.

The Agent Opportunity

What changed, and why an economist should care

Why Economists Should Care: Execution Is the Bottleneck

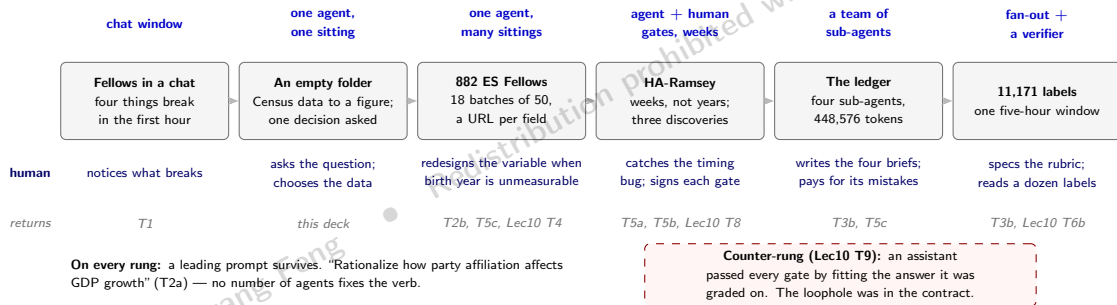
The bottleneck in applied research is often not ideas. It is execution.

- Turning a good question into clean code, diagnostics, tables, and figures is slow
- Many research tasks are repetitive: reshape data, rerun specifications, trace sources, refactor scripts, update plots
- Agentic tools compress the time from *idea* to *first defensible result*
- That raises the return to the skills economists uniquely provide: question design, institutional knowledge, validation, and judgment

The promise is not “AI does research for you.” It is faster implementation and iteration around a question you still define and defend. The rest of the lecture supplies what that takes: how an agent runs (9.2), how to build and call one (9.3), how to design and validate one (9.4), and how to run, check, and govern a whole project (9.5).

The Ladder: From a Chat Window to a Verified Swarm

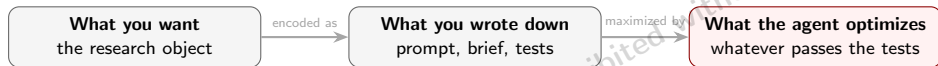
Every example in this lecture sits on one rung of the same ladder. Each rung buys reach and scale — and moves the economist's job from doing the work to checking it.



Not on the ladder: a vendor-reported ten-thousand-agent proof run, still disputed — a dated sidebar in T3b and Appendix TA.

Delegating to an Agent Is a Principal–Agent Problem

Economists already own the theory behind this lecture. Hand a task to an agent and you become a principal: you see what comes back, not how it was made.



Frictions you already know

- **Hidden action:** you observe the output, not the process (Holmström 1979)
- **A proxy objective:** a measure that becomes a target stops measuring (Goodhart 1975)
- **Relative prices moved:** generation got cheap; verification did not (Tao, ICM 2026)

So Lecture 9 is the principal's toolkit

- 9.2 know what you pay for: tokens, context, reach
- 9.3 write the contract: one job description per agent
- 9.4 audit it: planted defects, repeated trials
- 9.5 monitor the action: gates, traces, red lines

"Agent" means three things in this course: an LLM in a loop (here), a household in Aiyagari or HA-Ramsey, and the agent of contract theory. Evidence that this lens bites: Lec10 T9, where an assistant passed every gate by fitting the answer.

From Chatbox to Agent Team, 2022–2026

Four years, five capabilities: answering, calling tools, sharing plumbing, acting on your files, and running without you watching.



The model improved every year. What changed research practice was where the loop runs and what it can touch. Each step lowered the cost of doing the work — none lowered the cost of checking it.

Dates as of September 2026; sources in Appendix TA, which also explains MCP.

Story 1 — HA-Ramsey: Weeks, Not Years

Proper use of an agentic workflow turned one published solver into a new research program.

Seed: a working, validated representative-agent (RA) “deep Ramsey” neural solver — the complete code behind Feng–Han–Zhu (2026).

Target: the heterogeneous-agent (HA) Ramsey model of Chien–Feng–Wen (2027) — the entire paper: idiosyncratic risk, borrowing constraints, a 4–5D state.

Outcome: a full HA research codebase *plus* three methodological discoveries that surfaced from the AI iteration itself — α -shapes for non-convex feasible sets, the admissible set as a viability kernel, and a Lyapunov-type penalty for saddle-path selection.

- The AI supplied **throughput** — LaTeX drafting, PyTorch translation, diagnostics, 32+ sweeps; “time from seed to working solver: **weeks, not years**; the binding constraint was economist judgment at the milestones, not implementation labor”

Why show this first: this is what “using it properly” buys. Not autocomplete — research.

Full case with the economics and the figures: Lec10 T8, later this morning. Source: HA-Ramsey case-study package.

Story 2 — 882 Fellows: A Dataset You Could Build This Week

The second thread is deliberately ordinary: build a credible dataset about the economics profession from the public web.

- **Task:** career metadata for all 882 current Econometric Society Fellows — election year, PhD year, research fields, career stage
- **Seed:** the official Society roster, split into 18 batches of 50; every filled field must carry a source URL from an official or institutional page
- **What makes it hard:** roster control, source discipline, state across sessions, auditable provenance — *bookkeeping, not intelligence*
- **What happened:** the agentic workflow scaled the search massively; the honest headline was about what still could *not* be measured (T5c tells that part)

Why two stories: HA-Ramsey shows the ceiling of the method. The fellows dataset is the floor you can stand on immediately — in Exercise B, you build one batch of 50 yourself.

Full case with the coverage results: Lec10 T4, later this morning.

Chat, Copilot, Agent

Where the loop runs, and what it can touch

What Modern Web Chat Already Does Well

As of mid-2026, a strong web-chat product is already useful for economists.

- Brainstorming model setups, empirical strategies, review rubrics, and robustness checks
- Summarizing papers, referee reports, and lecture notes
- Inspecting uploaded files and explaining code snippets
- Sometimes browsing the web or running limited sandboxed tools
- Helping you draft a specification before you open your editor

So the weak baseline is not “a browser cannot do anything.” The weak baseline is a *single chat session without a durable harness*.

What Breaks Without a Harness: The Fellows Test

Try the fellows task in a plain chat window. It is a job of many sittings — and four things break within the first hour.

1. **Durable context is weak.** The task spans days of separate sessions. The coding rules you settle today — which sources count, how to record a missing PhD year — are not reliably remembered by tomorrow's fresh chat
2. **Reusable workflow is weak.** Your checklist lives in prompts you retype (slightly differently) each time, not in a file the tool reads automatically
3. **Controlled execution is weak.** The chat works on copies you upload, inside its own sandbox; it cannot rerun a check against the files in your project — *you* become the copy-paste machine between the model and your folder
4. **Audit trail is weak.** Six weeks later: which source URL backed the PhD year of fellow #312? A scrolled-away transcript cannot answer field-level questions

That is the real contrast. Agents are not magic because they are “smarter.” They are useful because they operate inside a structured project with files, tools, and repeatable workflows.

Same Model, Different Reach: Where the Loop Runs

Same model. What changes is where the loop runs, what it can touch, and what survives.

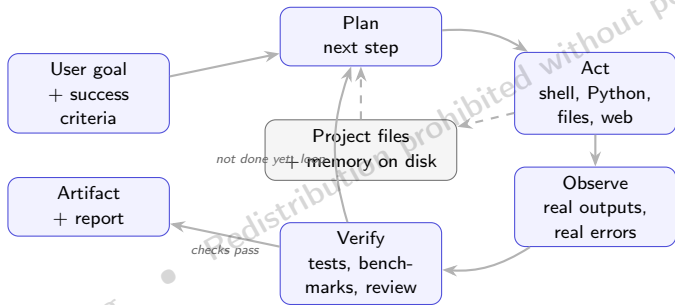
	Web chat	IDE assistant	Terminal agent
Loop runs	on the vendor's servers, in a sandbox	inside your editor	on your machine or your cluster
Can touch	files you upload; sandboxed code	the project open in the editor	your files, your Python, R or Stata, your shell — within your permissions
Persists	the conversation	open files and their edits	files on disk: code, data, notes, standing instructions
Presses run	you, one exchange at a time	you, or its agent mode	the agent, until a check stops it
Economist's use	design the specification	clean one regression file	run the whole pipeline

These are rungs, not brands. Web chats now run sandboxed code, and IDE assistants ship agent modes that run commands. Terminal agents — Claude Code (our running example), Codex CLI, Gemini CLI — differ in *reach*, not intelligence: they work where your data and compute already live.

T2a opens the loop; T5a returns to tool choice; dated product details: Appendix TA.

The Agentic Loop: Plan, Act, Observe, Verify, Repeat

An agentic workflow is not one long generation. It is a control loop wrapped around tools, files, and checks.



- **Orchestrator:** the loop deciding what to do next — plan, act, observe, verify
- **Memory:** saved project state on disk, not just chat history (dashed arrows: the loop reads it and writes it)
- **Verifier:** tests, benchmarks, or review checks deciding whether output survives

What Makes Something “Agentic”?

- **Tool use** — invokes pandas, curl, R, statsmodels, the shell, whatever the task requires
- **Autonomy** — plans a multi-step empirical workflow without re-prompting.
e.g., “download FRED → clean → regress → plot → save table” as one command
- **Iteration** — observes that a regression failed, identifies the bad column, fixes the formula, reruns
- **Persistence** — remembers your project conventions across sessions (variable names, preferred packages, validated benchmarks)

The features are cumulative. Tool use without autonomy = a glorified interactive console (a REPL — *read-eval-print loop*, like the Python `■>` prompt). Autonomy without iteration = a script that dies on the first error. You need all of them for “agentic.”

The Landscape in Three Frames

Patterns, costs, and how long agents really work

Four Orchestration Patterns

Production agent systems are built from a small set of recurring control structures.

Pattern	Shape
Orchestrator–worker	a lead agent decomposes a task and dispatches workers; <i>dominant in production</i>
Planner–executor	one component plans, another executes step by step
Reflection	the agent critiques and revises its own output before returning
Handoff	control passes between specialized agents (e.g. triage → specialist)

The agent team you will build in Part 9.3 and run as an RA team in T5b is an **orchestrator–worker system**: the main agent orchestrates; code-reviewer, domain-reviewer, and verifier sub-agents are the workers. The landscape just adds vocabulary and tooling around the same loop.

The Multi-Agent Cost–Benefit Decision

More agents is not free. Treat it as marginal value vs. marginal cost.

- Anthropic reports its multi-agent research system beat a single-agent baseline by **+90.2%** on its own research eval. . . **[hype]**
- . . . at roughly **15× the token cost [hype]**
- So multi-agent pays off when the task is broad, parallelizable, and the answer's value is high relative to tokens
- It is wasteful for narrow, sequential, or cheap tasks

[hype: vendor-internal eval] — a self-defined research task, not an external suite. Read the +90.2% as directional, not portable.

Economist framing. You are choosing a point where marginal research value = marginal token (and latency, and debugging) cost. The Aiyagari sub-agent review in T5b is justified on *wall-clock*, not because “more agents are better.”

How Long Can an Agent Actually Work? Measure, Don't Trust Marketing

Vendors say “works for hours” or “runs for days.” METR gives a measured alternative.

- METR's *task-length-doubling* result: the task length an agent completes at **50% reliability** has doubled about every **7 months** since 2019, and faster lately: about **4 months** from 2023 on, about **3** from 2024 on (Time Horizon 1.1, Jan 2026). **[solid]**
- The horizon is the length of task — timed on skilled humans — that an agent finishes at that reliability
- Model-by-model minutes change with every release: see Appendix TA (dated), not this slide

Metric fragility. The number is tied to a **50% reliability** threshold and a specific task suite; the horizon at 80% reliability is much shorter, and it shifts across suites. A measured-but-fragile number still beats a vendor claim.

[hype counterpoint] product pages advertising “multi-hour” or “multi-day” autonomy — no reliability threshold stated.

Research translation: the right question is not “can it run for hours?” but “at what reliability, on what task distribution, validated how?” *The rest of the landscape, dated: Appendix TA.*

What This Means for Research

Four patterns, and what stays human

Four Research Patterns Across the Lec10 Cases

Pattern	Lec10 case(s)	Main risk if you use the wrong workflow
Reusable skill	Case 1: referee-style paper review	Generic comments, hallucinated quotes, missing quality gate
Empirical pipeline	Case 2: MEPS harmonization and figures	Wrong crosswalk, dropped survey weights, silent validation failure
Public-web dataset	Case 3: fellows metadata with field-level provenance	Weak sources, hidden missingness, no audit trail
Full-lifecycle collaborator	Cases 4–7: collaborator, 4-day sprint, HA-Ramsey, MPE shortcut	Polished output built on weak framing, loose claims discipline, or a gate that can be gamed

Four patterns organize Lec10's seven cases — our two threads are instances: the fellows dataset (Case 3) and HA-Ramsey (Case 6, the full-lifecycle pattern at research scale). Case 7 is the cautionary one: every gate passed, and the model was not solved.

Where RAG fits. Agentic RAG (Lec08 T5) differs from single-shot RAG: the agent decides *dynamically*, via the same tool-call mechanism, whether and when to retrieve. RAG is one tool among many that the agent can compose.

What Remains Human

The gain is not that judgment disappears. Disciplined execution becomes cheap — which raises the return to judgment.

You still own

- the question and its identification logic
- the success criteria and the benchmark
- the coding and source rules
- knowledge of the data, the model, the institutions
- the interpretation — and its defense in a seminar

The agent accelerates

- code generation and refactoring
- file handling, scripts, and test runs
- decomposition you can prompt for
- reruns and iteration on visible errors
- draft diagnostics and reports

Research version of the slogan: the agent can be a strong RA; you remain the principal investigator.

Hold on to the left column: T5b turns it into a signed division-of-labor contract, and T5d into seven tests and a job description.

Bridge: Open the Hood

You have seen what agents do and what proper use produced. The question now changes from “what is this?” to “how does it actually run, and how do I start?”

- **T2a** opens the loop in machine language: one JSON line, a session that is only a list, and a real research question traced call by call
- **T2b** puts the loop on your machine: install, files, commands, permissions, a first session
- **The economist's version of Part 9.2:** what exactly am I paying for — and what can the agent see when it acts?

Dated detail on the wider landscape: Appendix TA.

Arc: 9.1 See → 9.2 Under the Hood → 9.3 Build → 9.4 Design & Validate → 9.5 Run & Govern.